Data Structures 1 – Winter 2025

Assignment A2

General Rules

Item	Details
Deadline	**December 6, 11:59 p.m.**
Late Penalty	10 pts if submitted Dec 7, 20 pts if submitted Dec 8, 30 pts if submitted Dec 9 **No submissions accepted after Dec 10**
Group Size	Pairs (two students)
Oral Defense	You may be asked to explain your solution orally. Failure to answer satisfactorily implies a grade 0 in each exercise involved in the discussion.
Academic Integrity	Any detected plagiarism or copying will be reported to the UG office.
Questions	None allowed during the exam period.
Language & Environment	Java (must compile in VS Code and pass all provided tests).
Deliverables	One **zip** file containing all Java source files (*wet part*).One **PDF** file with all written answers (*dry part*). Please do not use handwriting, except for figures.

---

1. [5p] Suppose that we have numbers between 1 and 1000 in a binary search tree, and we want to search for the number 370. Which of the following sequences could not be the sequence of nodes examined? (notice it may be more than one)

a. 2, 252, 401, 398, 330, 344, 397, 369, 371.

b. 924, 220, 902, 244, 898, 258, 362, 370.

c. 2, 399, 387, 219, 266, 382, 381, 278, 370.

d. 925, 100, 202, 903, 240, 900, 230, 370.

e. 935, 278, 347, 621, 299, 392, 358, 369, 370.

2. [10p] Show that the recursive version of DELETE algorithm for AVL trees runs in O(log n).

- Hint: you need to show the complexity of all functions that are invoked by DELETE.

3. [20p] Given an AVL designed using following data structure:

```
AVL: <root: AVLNode>
AVLNode: <int value, AVLNode left, AVLNode right, int height>
```

a. **[2p]** Provide the representation invariant for this data structure to effectively represent an AVL.

b. **[3p]** Provide the pseudocode of the iterative (non-recursive) versions of algorithms to insert, delete and search elements in the AVL.

c. **[6p]** Prove that the previous algorithms preserve the representation invariant (it is true before and after the execution of each of the 3 methods).

d. **[7p]** Implement the algorithms in Java, including the method boolean `repOK()` that checks the representation invariant, and provide meaningful tests, including test that checks `repOK()`.

4. [10p] Given a parameter i,

a. Provide the pseudocode of an algorithm that generates the ith Fibonacci tree.

b. Implement the algorithm in Java and provide meaningful tests.

c. Write a method in Java that prints the balance factor of each node of the ith Fibonacci tree.

5. [15p] Heaps over binary tree and over arrays

For the next two items you can either represent the binary tree using the mathematical representation (i.e., `BT = Nil | <root, BT, BT>`) or the in memory representation (i.e., `BT = <root: Node>, Node: <data, left: BT, right: BT>`).

a. **[5p]** Write an algorithm that converts a binary tree into an array. What are the preconditions over the binary tree such that the resulting array has the max-heap property?

b. **[5p]** Write an algorithm that converts an array into a binary tree. What are the preconditions over the array such that resulting binary tree has the max-heap property?

c. **[5p]** Implement both operations in Java and provide meaningful tests.

6. Extending the Basic Blockchain System – 40 pts (wet)

Extend the Simple Blockchain to become an **Advanced Blockchain (AB)** to incorporate efficient balance management, transaction prioritization using fees and faster block retrieval.

Core Modifications & Additions

Fee-Based Transaction System with a Priority Queue The new system will incorporate a positive integer `fee` which, for simplicity, will be paid with BB-coins. For transactions that finalize successfully the total `fee` offered by the sender will be consumed, but for reverted transactions only half of the `fee` will be consumed and the other half will be reimbursed.

More importantly, transactions within blocks are now prioritized by `fee`. That has two implications:

- There will be a `Transaction Pool` (a priority queue) where all transactions will be stored first in the pool, waiting to be placed in the Blockchain.
- The transactions within the pool will be prioritized by a combination of order of insertion and `fee` given by the formula:

  ```
  priority = fee + 1/(order of insertion)
  ```

  where maximum priority means first to be attended. Note that `priority` should be of type `double`. Hint: For the order of insertion you may use a counter of all the transactions that were inserted.
- A block miner will extract transactions from the pool and insert them in the current block.
  - Transactions within the block, when executed, will be prioritized only by its `fee`. The transaction with maximum `fee` should be executed first in the block.

Balance Management Balance lookup and update are now O(log A) operations (with A the number of addresses in the blockchain).

The balance now includes a `totalSupply` balance which initially coincides with the amount of BB-coins of the address "0" in the genesis block. This `totalSupply` must be always the sum of all accounts and will be maintained through all balance updates.

Faster Block Indexing

Now blocks can be retrieved in O(log n).

Your implementation task:

Enhanced Blockchain Class `ABlockChain`: New Methods: - `requestTransaction(String from, String to, int amount, int fee)` - Adds transaction to the transaction pool, using the fee and the insertion order for determining the priority. - **Complexity**: O(log TP) where TP is the number of transactions in the pool.

- `mineBlock()`: mines a block with transactions.
  - Selects transactions from the transaction pool based on priority to fill the current block.
    - If the block becomes full, invokes the method `processCurrentBlockAndStartNewBlock` to process the transactions and update the balances and returns true.
    - Otherwise returns false, leaving the currentBlock in the last state (half full or empty).
  - **Complexity**: O(TB* (log TP + log A)) where TB is the number of transactions per block and A the number of addresses with balance.
- `processCurrentBlockAndStartNewBlock`: process a full block, by executing its transactions and updating balances.
  - The transactions in the block are executed in order determined only by the `fee`.
  - A transaction succeed when the sender has enough balance to pay the `amount` and the transaction `fee`.
  - Successful transactions will update the balance of the sender and receiver accordingly. The `fee` is deducted from the balance of the sender and transferred to address "0".
  - Non successful transactions will be marked as reverted and half of the `fee` will be deducted to the sender and transferred to address "0", only if the sender have enough coins to pay the `fee`. Otherwise the `fee` won't be deducted.
  - **Complexity**: O(TB*log A) where TB is the number of transactions per block and A the number of addresses with balance.
- `getBlockByNumber(int number)`: Obtains the block using the block number.
  - **Complexity**: now is O(log B) where B is the number of blocks.
- `getTransactionPoolSize()`: Returns the current size of the Transaction Pool.
  - **Complexity**: O(1).
- `getSuccessfulTransactionsCount()` and `getRevertedTransactionsCount()`"Return total number of successful and reverted transactions
  - **Complexity**: O(1).
- `getReturnedFees()`: Return the total amount of BB-coins in fees returned due to reverted transactions.
  - **Complexity**: O(1).
- Provide a new `repOK()` covering the new representation invariant.

Recall: Transactions in teh pull use a priority determined by `fee` and the order of insertion. Transactions in blocks are processed by using only the `fee` as priority.

Improved Balance Operations:

- `getBalance` and `updateBalance`:
  - * **Complexity**: O(log A) where A is number of addresses
- `repOK()` that ensures that the `totalSupply` is the sum of all accounts.

Block class It remains the same but name transactions use a Priority Queue, where the priority is given by the `fee`. More `the`fee' implies more priority.

New operations:

- `getTransactions()` returns an array of transactions, in the order given by the priorities.
  - **Complexity**: O(T log T). where T is the number of transactions in the block.
- Operation no longer needed: `removeTransaction()`.

New subclasses for modeling transactions You need to implement two new classes: `TransactionWithFee` that extends the `Transaction` class and `TransactionWithOrder` than extends `TransactionWithFee`.

- They use the method `compareTo` for defining the transaction ordering policy.

You may use these different kind of Transactions for the priorities in the transaction pool and within each block in the blockchain.

Final Notes

- See general rules.
- **Do NOT use** Java's built-in classes (e.g. `TreeMap`, `HashMap`, or `LinkedList`).

- Provide **your own data structures**. You can use all code we used in the tutorials 1,2,3,4,5,6.
- Supply **meaningful unit tests** covering edge cases.

- Justify **complexity analysis** for every method.

- Code must be **readable**, **correct**, and **pass all supplied tests**.
- Recommendation: implement toString methods for all classes to help visualization during the task of debugging.